COMP 018: Database Administration - Activity 4 Progress Report

Access Control, Database Users, Roles, Privileges, and Security Governance

Project Identification & Metadata

Course	COMP 018 - Database Administration
Evaluating Group	Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository	https://github.com/Nycri/brewpos
Target Project Monitored	CampusSafe: Campus Hazard & Safety Reporting System
Target Project Group	Group 7 (Alba, Arreza, Orpesa, Robiso, Santos)
Target Project Repository	https://github.com/jeffersonorpesa2-ops/grp07-campussafe
Primary Database	grp07_campussafedb (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	grp07_campussafedb_restore_test

ACTIVITY 4: Database Users, Roles, Privileges, and Access Control

1.1 Access-Control Plan & Architecture

In default database deployment configurations, applications frequently connect using an unrestricted superuser account or standard database owner credential. When an application runtime account holds blanket permissions across all tables, any application vulnerability, such as SQL injection or an exposed administrative endpoint, enables malicious actors to purge incident record histories, remove audit logs, or extract user password hashes. This scenario presents a direct violation of the Principle of Least Privilege.

For Group 7's CampusSafe system, a comprehensive role-based access control architecture was implemented to separate administrative infrastructure maintenance from application runtime operations. The structural hierarchy utilizes group roles without login capabilities to manage privilege collections, alongside individual login accounts assigned to their corresponding group roles.

Under this plan, the application runtime account holds standard read, insert, and update capabilities across active tables, but delete permissions are restricted exclusively to image attachment removals and work assignment reallocations. Row deletion remains strictly prohibited on core hazard reports,

report status histories, maintenance actions, and user profile records. In addition, the read-only auditing account receives query access for metric inspection but is denied access to user profile tables, preventing reporting mechanisms from accessing password hashes.

1.2 Access-Control Role Definitions and Privilege Structure

The application runtime user operates under a dedicated group role assigned to manage active hazard report submissions, status transitions, technician assignments, and attachment updates. Its database connection and schema usage are restricted to operational necessities, maintaining full selection, insertion, and update capabilities on functional tables while confining record deletion to attachments and assignments. This constraint ensures that even under an application-level breach, foundational incident logs and user profiles remain protected against unauthorized deletion.

The read-only auditor account operates through a compliance group role to review campus safety performance metrics and resolution timeliness. It maintains query access across operational logs while omitting read permissions on user credentials to safeguard authentication data. The backup operator account is assigned dedicated read permissions across tables and sequences to support logical dump generation without granting data modification or schema alteration rights. Administrative tasks, schema migrations, and recovery operations remain reserved for the database administrator superuser account.

1.3 Access-Control Configuration Summary

The configuration process established three non-login group roles representing runtime, read-only audit, and backup profiles, followed by the creation of matching login accounts with dedicated authentication credentials. Permissions were revoked from public roles across the database and public schema to enforce explicit access assignment. Connection privileges were assigned to the designated group roles, alongside specific schema usage rights. Explicit table grants were applied to each role according to operational requirements, and default schema privileges were updated to maintain these access definitions across newly created objects.

1.4 Verification Results: Runtime Account Testing

Testing confirmed that the runtime account successfully queries active hazard reports and performs valid insertion operations, advancing primary sequence counters. Updates to mutable attributes such as report priority levels process without error. File attachment deletion executes properly to support administrative attachment cleanup.

When deletion operations were attempted on hazard report records or user identity profiles, the database engine denied the execution, enforcing the narrow deletion policy. Data definition operations, including table drop commands, were rejected due to missing object ownership permissions.

1.5 Verification Results: Read-Only Auditor Account Testing

Inspection queries executed under the auditor account successfully returned report metrics joined across categories and campus location tables. When querying user profile tables containing credentials, access was denied by the database security layer.

Attempted modification operations, including record insertions into category tables, status priority updates on report records, and row deletions, were uniformly blocked. These results verify that the read-only audit role maintains reporting capabilities without risk of record alteration.

1.6 Verification Results: Cross-Database & Administrative Boundary Testing

Administrative isolation testing showed that creation commands for new databases were denied due to missing creation privileges. Connection attempts to unrelated databases on the server instance failed because public connection permissions were revoked, restricting access strictly to authorized database targets.

1.7 Issues, Fixes, and Feedback to Owners

During deployment, environment configuration required establishing a user-level configuration file to load required PHP extensions for database communication. Local launcher batch scripts were updated to automatically detect configuration paths, enabling environment execution without manual path definitions. Database account credentials were aligned with local environment properties to synchronize password hashes across demo accounts.

Feedback provided to Group 7 highlighted the effective relational structure of the CampusSafe system, noting its enforcement of status workflows and audit logging through relational constraints. Recommendations included adding configuration path checks in initialization scripts to streamline evaluation on standalone PHP setups, as well as providing explicit user password update guidelines within database setup documentation.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 4)

Column Header	Value
Project	Group 7 - CampusSafe: Campus Hazard & Safety Reporting System
Repository cloned	https://github.com/jeffersonorpesa2-ops/grp07-campussafe, Branch: main
Database setup result	Success (Configured user php.ini with pdo_pgsql, verified PostgreSQL 18 on port 5432, loaded 9 tables and seed data).
Access-control folder link	https://github.com/Nycri/brewpos/tree/main/access-control
Access-control status	Completed
Feedback sent to owners	(05/11/2026)
PDF progress document	Activity4_Progress_Report_Group7.pdf
Last updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.